Introduction

We attempt to replicate the stabilization features of modern cannons using a gyroscope and accelerometer communicating to the Pico through an I2C library that will keep whatever object is on it stable. The pico reads the gyroscope data, computes the counter-correction angles, and drives a servo motor to keep the object stable. The FPGA will get the information from the Pico using SPI and work with the RGB LCD display to show the current angle of the object and the correction amount in real time.

Elements of Complexity

We implement a custom 3-byte SPI protocol with a byte-counting state machine and dual flip-flop synchronizers to ensure reliable communication between the Pico and FPGA across different clock domains. The FPGA renders all graphics in real-time without a framebuffer, using a sine/cosine lookup table for needle positioning, a custom font ROM for text rendering, dynamic digit positioning to handle varying number lengths, distance-squared integer math for arc rendering, discrete tick marks placed at regular intervals, title text displaying "CS122 FINAL", and conditional color output that makes the needle red while gauge elements appear white. The Pico runs sensor fusion using a complementary filter that combines accelerometer and gyroscope data, with gyro calibration performed by averaging samples to remove bias offset. A PID control loop with integral anti-windup clamping, deadband handling to prevent servo jitter, and feed-forward control positions a servo using PWM with mapped pulse widths. The potentiometer auto-calibrates to map target angles from zero to ninety degrees, while a fixed real-time loop ensures deterministic updates. The system bridges dual clock domains through SPI communication, with the Pico handling sensor fusion and servo control while the FPGA manages all graphics rendering.

User Guide

This project’s inputs and outputs were very simple. You have a potentiometer which allows you to change the elevation of the “cannon" between 0 and 90 degrees from the horizon. To visually see the angle you’ve chosen, the LCD screen connected to the FPGA has a horizon indicator and a small text that tells the user their current desired output and the amount of correction the system is currently applying to get the desired

angle. There is the physical output, of course: the servo actuation. Another output not explicitly for the user but used by us developers was printf statements in the pico code which can be read by a serial monitor.

Launch instructions: Upon pushing the code to the Pico 2, you must twist the potentiometer to its minimum and maximum settings to calibrate the adc reading. You have approximately 3 seconds to do this, after which the Pico will take another 1.5 seconds calibrating the readings from the MPU6050. No action is necessary after pushing top.bit to the Icesugar.

Hardware Components Used

- Icesugar Pro FPGA
- 4.3 inch TFT LCD display RGB 40PIN 480X272
- Raspberry Pi Pico 2 W
- MPU 6050 Accelerometer/Gyroscope
- Arduino Kit Servo motor
- Breadboard
- Jumper Wires
- Potentiometer

Software Libraries Used

List any externally-provided software libraries that you used. A simple bullet-point list is fine: include 1-3 sentences to describe how you used each library and how it contributed to the project.

Libraries:

1. math.h: Used to calculate the math behind the PID controller which is essential to keeping the “cannon” stabilized.

Raspberry Pi Pico APIs (Non-default ones):

1. pico/time.h: Used to create timer based interrupts, allowing the CPU to sleep the remainder of the 20ms period after it has finished calculations before being woken again at the end of the period/beginning of the new period. This allowed me to create a control feedback loop with the exact same period as the servo PWM period and update the control signal accordingly.

2. hardware/i2c.h: Used to read the accelerometer and gyroscope values from the MPU6050, which is used to read the physical output of the system.
3. hardware/adc.h: Used to read the potentiometer which is the user's only form of input to the system.
4. hardware/spi.h: Initializes the Pico as an SPI master at 250 kHz in Mode 0 to transmit 3-byte packets containing the current angle and target angle to the FPGA. The spi_write_blocking() function sends the data synchronously while the Chip Select pin is driven low, ensuring the FPGA receives clean, framed transactions at the 50 Hz control loop rate. This protocol offloads all graphics rendering to the FPGA, allowing the Pico to focus on sensor fusion and servo control.
5. hardware/pwm.h: Allows for the configuration of a pwm system with a period relative to the clock period of the CPU. This allows us to actuate the servo.

Protocols Used

1. SPI - Used for communication between the Pico and FPGA to send angle and target data as 3-byte packets at 250 kHz. The Pico acts as master, sending control data, while the FPGA receives the values and renders the compass gauge.
2. I2C - Used to read accelerometer and gyroscope data from the sensor at 400 kHz. This two-wire protocol allows the Pico to retrieve orientation data for calculating the current angle.
3. PWM - Used to generate a 50 Hz control signal for the servo motor. The hardware PWM module provides precise timing for smooth servo movement.
4. ADC - Reads analog voltage from a potentiometer to set the target angle. The ADC converts the potentiometer position to a digital value, which is then mapped to a 0-90° range.
5. UART via USB - Prints debug information including filtered angle, desired angle, servo position, and PID values.
6. LVCMOS I/O Standard - Used as the signaling standard for SPI, PWM, ADC, and all digital I/O between the Pico and FPGA. This ensures voltage compatibility and signal integrity across devices.
7. PLL Clock Generation - Multiply the 25 MHz input clock and generate a stable pixel clock for LCD timing. This provides synchronization for the 480×272 display at 60 Hz.
8. RGB Parallel Display Interface - The FPGA runs the LCD using a parallel RGB interface with 5/6/5-bit color channels along with HSYNC, VSYNC, and DE control signals.

Requirements

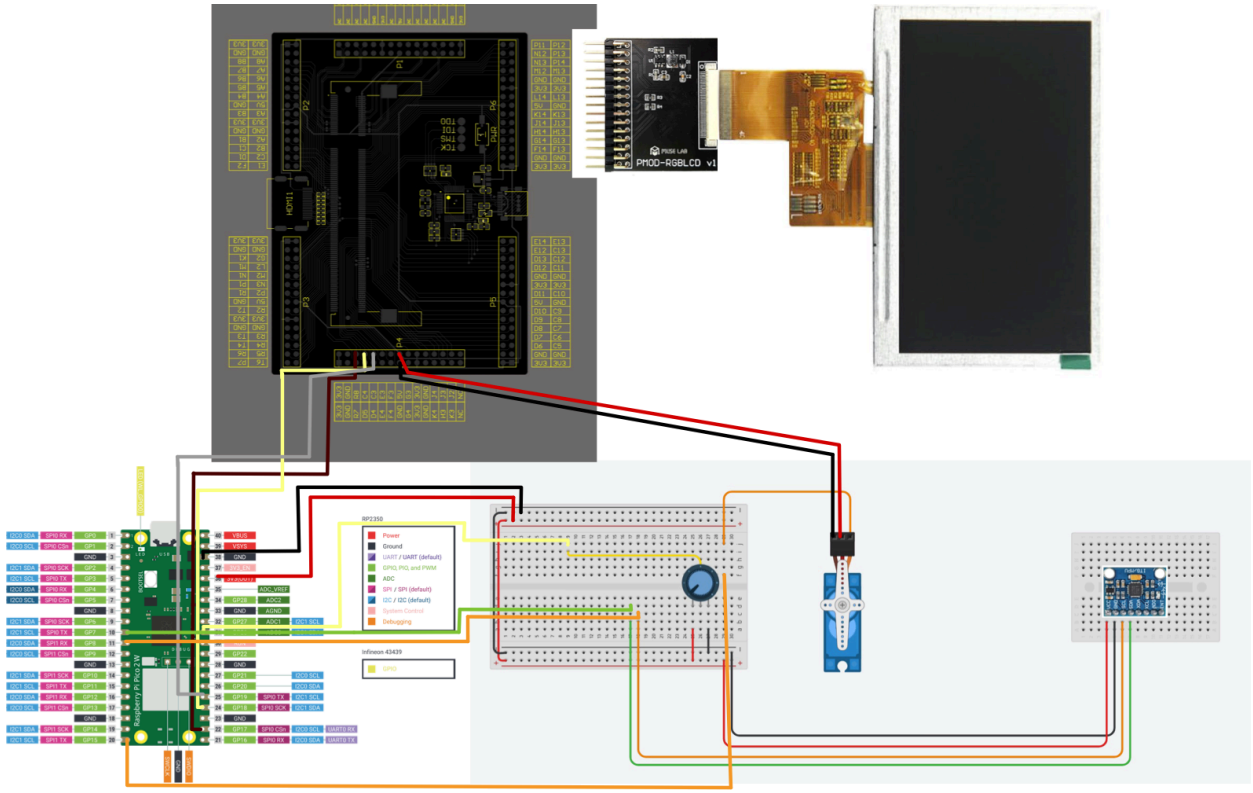
Icesugar-pro FPGA: The iCEsugar-Pro FPGA received angle data from the Pico over SPI, rendered the compass gauge graphics, and drove the LCD display. It handled all real-time tasks including needle rendering, text display, and LCD timing generation at 60 Hz. This allowed the Pico to only send simple angle values while the FPGA managed all graphics processing.

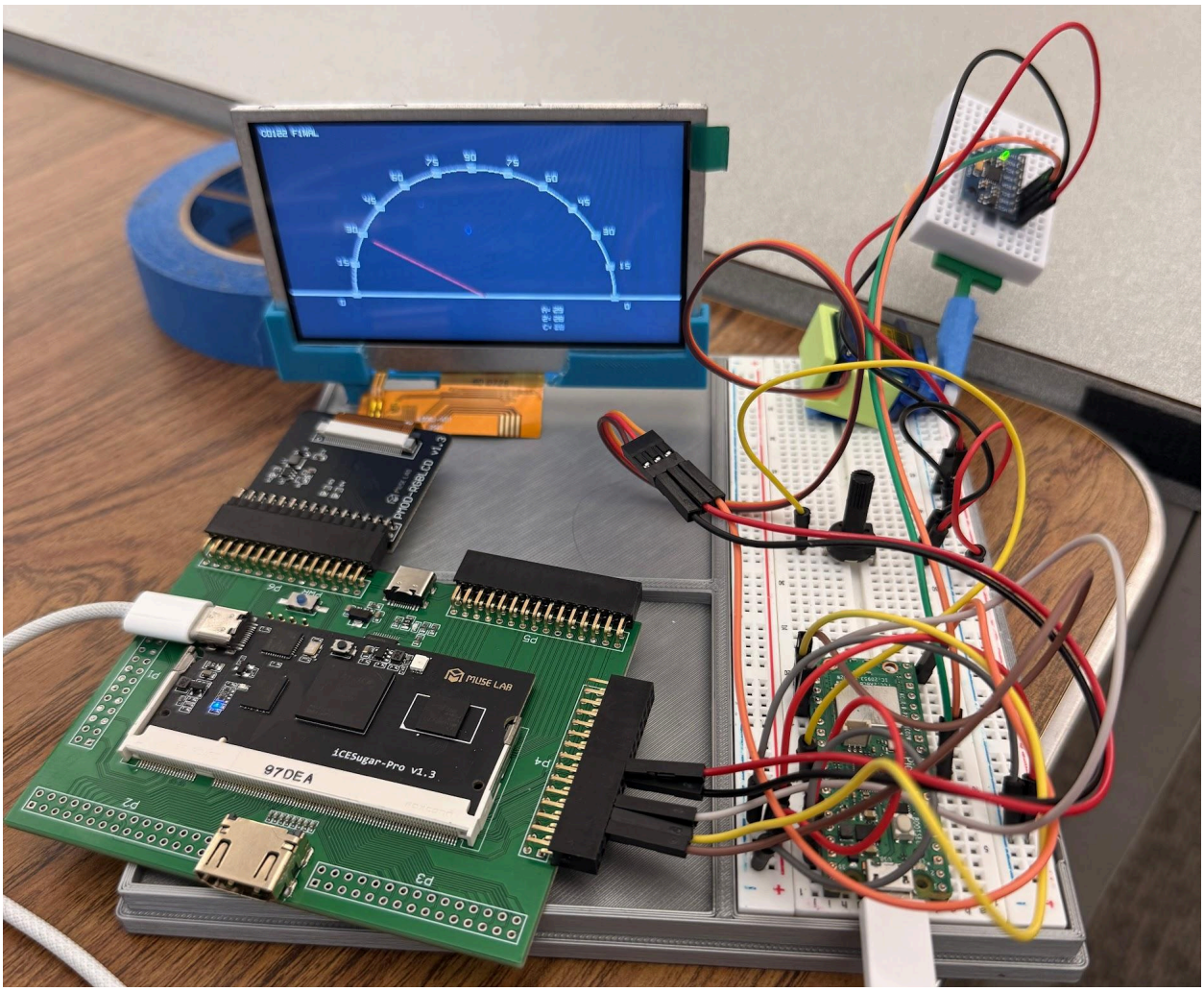
RGB LCD from the required lab components: Used to display an artificial horizon with a needle to point out the current desired angle to the user, and text that textually displays the same information.

Raspberry Pi Pico 2W: The Pi is the primary controller, and is connected to every peripheral directly or indirectly. It reads the values from the MPU6050 via I2C and does the calculations for the PID controller and then converts the calculations into a PWM signal for the servo. It then sends the value of the desired angle and the amount of correction to the Icesugar-pro via SPI.

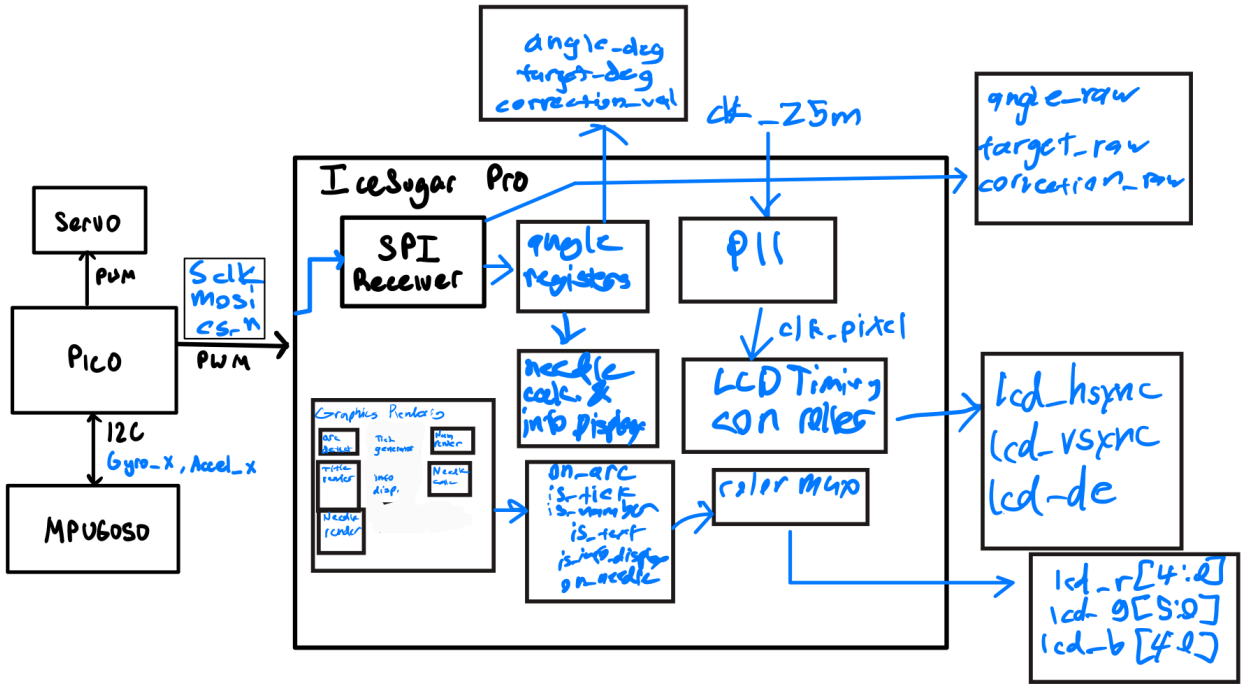
Event driven execution(timing call backs are a type of event driven execution): The system is running at a continuous 50Hz loop, where it does the calculations and then sleeps until exactly 20ms after when it woke up using the Pico’s built in timer library.

Wiring Diagram


PMOD-RGBLCD v1
Raspberry Pi Pico 2W
RP2350
Infineon 43439


PMOD-RGBLCD v1.3
MUSE LAB
1CESugar-Pro v1.3
97DEA

Design Diagram



AI Usage

Jonathan: I used primarily Gemini to help me outline what the Pico code should entail, what libraries and Pico packages I should use. For these packages I also used Gemini as a more user friendly form of documentation. Additionally, I used it to debug when I ran into bugs for the Pico behavior.

Josiah: I used AI to organize the top.sv file so that all the related functions would be in easy to find locations. I also had it figure out the functions for the circles, tick marks, and needle positions for the display. I also had it generate all the angle positions so I wouldn't have to do it manually.

Acknowledgements

List any resources used whether you used code from it or if it was just for information.

1. MPU 6050 Beginner Guide
2. Raspberry Pi Hardware API Documentation
3. Raspberry Pi Pico I2C MPU6050 Example
4. Bresenham's Line Generation Algorithm